

SẮP XẾP DỮ LIỆU

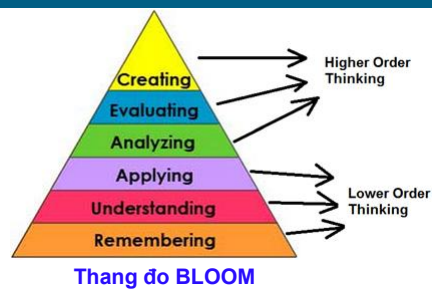


1



Mục tiêu

- Xác định và xử lý các giá trị bị khuyết.
- Vận dụng định dạng dữ liệu.
- Vận dụng chuẩn hóa dữ liệu.
- Chuyển đổi giá trị trong bộ dữ liệu.



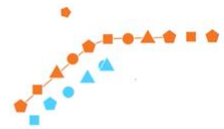
2

2



Nội dung

1. Tiền xử lý dữ liệu trước khi phân tích.
2. Xử lý giá trị bị khuyết.
3. Định dạng dữ liệu.
4. Chuẩn hóa dữ liệu.
5. Biến đổi giá trị số thành giá trị phân loại.
6. Biến đổi giá trị phân loại thành giá trị số.



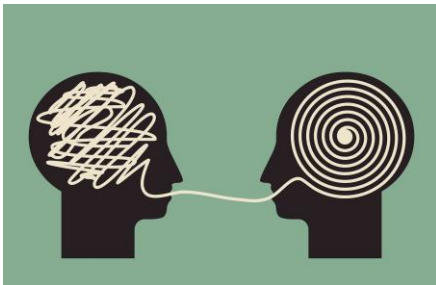
3

3



Sắp xếp dữ liệu - Data Wrangling

Tại sao phải sắp xếp dữ liệu?



4

4



1. Tiền xử lý dữ liệu

Quá trình chuyển đổi hoặc ánh xạ dữ liệu từ dạng “thô” sang một định dạng khác, để chuẩn bị dữ liệu trước cho phân tích.

(Joseph Santarcangelo, Ph.D., Data Scientist at IBM)

- Thuật ngữ tương tự:

- Data Cleaning
- Data Wrangling

” *80 percent of a data scientist's valuable time is spent simply finding, cleansing, and organizing data, leaving only 20 percent to actually perform analysis...*

IBM Data Analytics

5

5



1. Tiền xử lý dữ liệu

Nhiệm vụ chính

- Tìm và xử lý giá trị khuyết
- Định dạng dữ liệu
- Chuẩn hóa dữ liệu
- Chuyển giá trị số thành các giá trị phân loại
- Chuyển các giá trị phân loại thành giá trị số

6

6



2. Xử lý giá trị bị khuyết

Giá trị bị khuyết?

- Giá trị bị khuyết là giá trị bị thiếu (trống/rỗng) trong bộ dữ liệu (hoặc df)
- Xảy ra khi một mẫu nào đó thiếu giá trị của một thuộc tính (**hay feature**) nào đó.
- Thông thường, giá trị khuyết trong dữ liệu thô được thể hiện dưới dạng:
 - Dấu chấm hỏi “?”
 - Dấu chấm “.”, “Non”, “Null”
 - “N/A”, “NA”, “Nan”, “NaT”
 - Hoặc “0” (tùy tình huống)
 - Hoặc để “ô trống”.

	Attribute1	Attribute2	Attribute3	Attribute4
0	...		...	...
1	...	...	?	...
2	...	?	...	
3	?	...		N/A
...	...	N/A	...	...
n	...	...	0	...

...

7

7



2. Xử lý giá trị bị khuyết

Ví dụ giá trị bị khuyết

	symboling	normalized- losses	make	fuel- type	aspiration	num- of- doors	body- style	drive- wheels	engine- location	wheel- base	...	engine- size	fuel- system	bore	stroke
0	3	?	alfa-romero	gas	std	two	convertible	rwd	front	88.6	...	130	mpfi	3.47	2.68
1	3	?	alfa-romero	gas	std	two	convertible	rwd	front	88.6	...	130	mpfi	3.47	2.68
2	1	?	alfa-romero	gas	std	two	hatchback	rwd	front	94.5	...	152	mpfi	2.68	3.47
3	2	164	audi	gas	std	four	sedan	fwd	front	99.8	...	109	mpfi	3.19	3.40
4	2	164	audi	gas	std	four	sedan	4wd	front	99.4	...	136	mpfi	3.19	3.40
5	2	?	audi	gas	std	two	sedan	fwd	front	99.8	...	136	mpfi	3.19	3.40
6	1	158	audi	gas	std	four	sedan	fwd	front	105.8	...	136	mpfi	3.19	3.40
7	1	?	audi	gas	std	four	wagon	fwd	front	105.8	...	136	mpfi	3.19	3.40
8	1	158	audi	gas	turbo	four	sedan	fwd	front	105.8	...	131	mpfi	3.13	3.40
9	0	?	audi	gas	turbo	two	hatchback	4wd	front	99.5	...	131	mpfi	3.13	3.40

...

8

8



2. Xử lý giá trị bị khuyết

Phương pháp xử lý dữ liệu bị thiếu?

- Hiểu rõ nguồn dữ liệu đang có.
- Kiểm tra nguồn thu thập dữ liệu.
- **Loại bỏ giá trị bị thiếu**
 - Loại bỏ biến/thuộc tính
 - Loại bỏ nguyên mẫu dữ liệu
- **Thay thế giá trị bị thiếu**
 - Thay thế bằng giá trị trung bình (*average*)
 - Thay thế bằng tần suất (*frequency*)
- Dùng **thuật toán** để điền khuyết giá trị(*)

	Attribute1	Attribute2	Attribute3	Attribute4
0	...		...	...
1	...	...	?	...
2	...	?	...	
3	?	...		N/A
...	...	N/A	...	...
n	...	...	...	...

*Ref: <https://scikit-learn.org/stable/modules/impute.html>

9

9



2. Xử lý giá trị bị khuyết

Phương pháp 1: Tìm và loại bỏ các giá trị bị khuyết

`DataFrame.dropna(axis=0, how='any', thresh=None, subset=None, inplace=False)`

Parameters

axis {0 or 'index', 1 or 'columns'}, default 0

Determine if rows or columns which contain missing values are removed.

0, or 'index' : Drop rows which contain missing values.

1, or 'columns' : Drop columns which contain missing value.

how {'any', 'all'}, default 'any'

Determine if row or column is removed from DataFrame, when we have at least one NA or all NA.

'any' : If any NA values are present, drop that row or column.

'all' : If all values are NA, drop that row or column.

thresh int, optional

Require that many non-NA values.

subset array-like, optional

Labels along other axis to consider, e.g. if you are dropping rows these would be a list of columns to include.

inplace bool, default False

If True, do operation inplace and return None.

Returns

DataFrame

DataFrame with NA entries dropped from it.

10

10



2. Xử lý giá trị bị khuyết

Các phương thức khác

STT	Phương thức	Mô tả
1	<code>DataFrame.isna</code>	Xác định các giá trị khuyết
2	<code>DataFrame.notna</code>	Xác định các giá trị không khuyết
3	<code>DataFrame.fillna</code>	Thay thế giá trị khuyết
4	<code>Series.dropna</code>	Loại bỏ các giá trị khuyết
5	<code>Index.dropna</code>	Trả về các chỉ số không có giá trị NA / NaN

11

11



2. Xử lý giá trị bị khuyết

Áp dụng

`df.dropna(subset = ["price"], axis = 0, inplace = True)`

horsepower	peak-rpm	price
...	...	...
114	5400	16845
160	5300	19045
134	5500	NaN
106	4800	22470
114	5400	22625
102	5500	13950
...	...	...



horsepower	peak-rpm	price
...	...	...
114	5400	16845
160	5300	19045
106	4800	22470
114	5400	22625
102	5500	13950
...	...	...

Chú ý:

- `axis = 0` loại bỏ dòng
- `axis = 1` loại bỏ cột

12

12



2. Xử lý giá trị bị khuyết

Chú ý

```
df.dropna(subset = ["price"], axis = 0)
```

```
df.dropna(subset = ["price"], axis = 0, inplace = True)
```

Ref.: <https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.dropna.html>

...

13

13



2. Xử lý giá trị bị khuyết

Phương pháp 2: Thay thế các giá trị bị khuyết bằng giá trị khác

```
df.replace(  
    to_replace=None,  
    value=None,  
    inplace=False,  
    limit=None,  
    regex=False,  
    method='pad',  
)
```

Giải thích:

- **to_replace**: ký hiệu bị khuyết cần thay thế. Ví dụ: NaN, "?", ".", "
- **value**: giá trị mới. Ví dụ mean, mode...
- **inplace**: True/False

Ref: <https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.replace.html>

...

14

14



2. Xử lý giá trị bị khuyết

Áp dụng

```
mean = df["horsepower"].mean()  
df["horsepower"].replace(to_replace=np.nan, value=mean)
```

horsepower	peak-rpm	price
...	...	...
114	5400	16845
160	5300	19045
134	5500	21485
NaN	4800	22470
114	5400	22625
102	5500	13950
...	...	...



horsepower	peak-rpm	price
...	...	...
114	5400	16845
160	5300	19045
134	5500	21485
124	4800	22470
114	5400	22625
102	5500	13950
...	...	...

15

15



2. Xử lý giá trị bị khuyết

Dùng thuật toán

- Có thể dùng thêm mô-đun **preprocessing** trong **sci-kitlearn** để hỗ trợ tiền xử lý dữ liệu.
- Ref: <https://scikit-learn.org/stable/modules/preprocessing.html>

16

16



2. Xử lý giá trị bị khuyết

Bài tập

- Làm sao phát hiện (detect) các giá trị bị khuyết theo dạng ký tự đặc biệt?

Gợi ý: Sau khi chuyển thành NaN thì dùng hai phương thức:

- isnull()
- notnull()

Sau đó, thống kê số lượng giá trị bị khuyết.

=> Từ đó phân tích là nên loại bỏ (drop) hay thay thế (replace)?

Trong bài LAB cũng sẽ có bài tương tự như thế này.

	Attr ibu te1	Attr ibu te2	Attr ibu te3	Attr ibu te4
0	...		...	...
1	...	...	?	...
2	...	?	...	
3	?	...		N/A
...	...	N/A	...	...
n	...	...	...	...

...

17

17



2. Xử lý giá trị bị khuyết

Bài tập

- Hướng dẫn

	Attribute1	Attribute2	Attribute3	Attribute4
0	...		...	...
1	...	...	?	...
2	...	?	...	
3	?	...		N/A
...	...	N/A	...	...
n	...	...	...	...

	Attribute1	Attribute2	Attribute3	Attribute4
0	...		...	...
1	...	...	NaN	...
2	...	NaN	...	
3	NaN	...		NaN
...	...	NaN	...	...
n	...	...	...	...

...

18

18



2. Xử lý giá trị bị khuyết

Bài tập

- Tìm thêm một số giải pháp khác cho việc phát hiện các giá trị khuyết.
- Dựa vào các bộ dữ liệu trong các bài tập trước, dùng các phương pháp đó minh họa.
- Tìm hiểu **Nearest neighbors** của **KNNImputer** trong **sklearn** để điền giá trị khuyết.

...

19

19



2. Xử lý giá trị bị khuyết

Gợi ý

```
import numpy as np
from sklearn.impute import KNNImputer
```

```
X = [[1, 2, np.nan],
      [3, 4, 3],
      [np.nan, 6, 5],
      [8, 8, 7]]
```

```
imputer = KNNImputer(n_neighbors=2)
```

```
imputer.fit_transform(X)
```

```
array([[1. , 2. , 4. ],
       [3. , 4. , 3. ],
       [5.5, 6. , 5. ],
       [8. , 8. , 7. ]])
```

...

20

20



3. Định dạng dữ liệu

Giới thiệu

- Dữ liệu thường được thu thập từ những nơi khác nhau và được lưu trữ ở các định dạng khác nhau => nên không nhất quán.
- Chuyển đổi dữ liệu dựa theo một **tiêu chuẩn tổng quát chung**, và cho phép người dùng so sánh ý nghĩa giữa chúng. (~ *qui về một “đầu mối” chung*)
- Ví dụ, cùng thể hiện địa chỉ là “Tp. Hồ Chí Minh” nhưng lại có nhiều thể hiện:
 - Hồ Chí Minh
 - Ho Chi Minh
 - Tp. HCM
 -

...

21

21



3. Định dạng dữ liệu

Ví dụ

Ren_luyen	Tam_tru	diem
...	...	...
78	KTX	5.7
87	Ky tuc xa	8.6
78	Túc xá	2.7
98	KTX	8.5
100	Kỷ túc xá	9.3
99	Kỷ túc	7.5
...	...	...

Chưa định dạng

- Gây nhầm lẫn
- Khó tổng hợp
- Khó so sánh

Ren_luyen	Tam_tru	diem
...	...	...
78	Kỷ túc xá	5.7
87	Kỷ túc xá	8.6
78	Kỷ túc xá	2.7
98	Kỷ túc xá	8.5
100	Kỷ túc xá	9.3
99	Kỷ túc xá	7.5
...	...	...

Đã định dạng

- Rõ ràng
- Dễ tổng hợp
- Dễ phân biệt

...

22

22



3. Định dạng dữ liệu

Áp dụng

- Chuyển đổi đơn vị tính “mpg” sang kiểu mới “L/100km” cho bộ dữ liệu.
- 1 MPG = 235,2 Lít mỗi 100 kilômét [l/100km]

```
1 df['city-L/100km'] = 235/df['city-mpg']
```

```
1 df[['city-mpg', 'city-L/100km']]
```

	city-mpg	city-L/100km
0	21	11.190476
1	21	11.190476
2	19	12.368421
3	24	9.791667
4	18	13.055556
...	...	...

24

24



3. Định dạng dữ liệu

Định dạng lại kiểu dữ liệu (Sai kiểu dữ liệu gốc)

- Đôi khi bị sai về kiểu dữ liệu của một trong bộ dữ liệu, chủ yếu là trong dữ liệu thô.

```
1 df['price']
```

```
0    13495
1    16500
2    16500
3    13950
4    17450
...
```

```
200   16845
201   19045
202   21485
203   22470
204   22625
```

Name: price, Length: 205, dtype: object

```
1 df.dtypes
```

```
symboling          int64
normalized-losses  object
make              object
fuel-type          object
aspiration         object
num-of-doors       object
body-style         object
drive-wheels       object
engine-location    object
wheel-base        float64
length            float64
width             float64
height            float64
curb-weight        int64
engine-type        object
num-of-cylinders   object
engine-size        int64
fuel-system        object
bore              object
stroke            object
compression-ratio  float64
horsepower         object
peak-rpm          object
city-mpg           int64
highway-mpg        int64
price             object
dtype: object
```

25

25



3. Định dạng dữ liệu

So sánh kiểu dữ liệu trong Python và Pandas

- Có nhiều kiểu dữ liệu trong Pandas. Cụ thể:
 - Objects: "A", "hello"...
 - Int64: 1, 2, 3, 4, 5...
 - Float64: 10.5, 23.56, 56.9...
 - ...

Bài tập nhỏ: Tìm hiểu thêm các kiểu dữ liệu khác trong Pandas?

26

26



3. Định dạng dữ liệu

Chỉnh sửa đúng kiểu dữ liệu

Phương pháp	Mô tả
<code>dataframe.dtypes</code>	Kiểm tra kiểu dữ liệu
<code>dataframe.astype('data type')</code>	Chuyển kiểu dữ liệu

- Chuyển kiểu dữ liệu **object** cột "price" sang kiểu số nguyên (**int**)
`df['price'] = df['price'].astype('int')`

27

27



4. Chuẩn hóa dữ liệu

Giới thiệu (Chuẩn hóa dữ liệu $\Leftrightarrow$ Data Normalization)

- Đồng nhất giá trị của các biến trong các phạm vi giá trị khác nhau thành cùng miền giá trị.

```
1 df[['length', 'width', 'height']]
```

	length	width	height
0	168.8	64.1	48.8
1	168.8	64.1	48.8
2	171.2	65.5	52.4
3	176.6	66.2	54.3
4	176.6	66.4	54.3
5	177.3	66.3	53.1
6	192.7	71.4	55.7
7	192.7	71.4	55.7
8	192.7	71.4	55.9

scale	[150,250]	[50,100]	[50,100]
impact	large	small	small

⇒ Sau khi chuẩn hóa xong thì đặc trưng của các mẫu không thay đổi.

...

28

28



4. Chuẩn hóa dữ liệu

Ví dụ

Hoc_phi	Ren_luyen	diem
...	...	...
12000000	78	5.7
19000000	87	8.6
18500000	78	2.7
13700000	98	8.5
19700000	100	9.3
16700000	99	7.5
...	...	...

Non-normalized

- “Hoc_phi”, “Ren_luyen”, “diem” khác miền dữ liệu
- Khó so sánh. “Hoc_phi” ảnh hưởng nhiều hơn



Hoc_phi	Ren_luyen	diem
...	...	...
0.12	0.78	0.57
0.19	0.87	0.86
0.185	0.78	0.27
0.137	0.98	0.85
0.197	1	0.93
0.167	0.99	0.75
...	...	...

Normalized

- Cùng miền
- Ít ảnh hưởng

...

29

29



4. Chuẩn hóa dữ liệu

Lý do chuẩn hóa dữ liệu

- Phục vụ cho thuật toán máy học
- Giảm ảnh hưởng của giá trị ngoại lai
- Dễ so sánh
- Cải thiện hiệu suất mô hình (Gradient Descent hoạt động tốt khi được chuẩn hóa)
- Trực quan hóa dữ liệu: hiểu rõ hơn về phân phối của dữ liệu và phát hiện xu hướng hoặc cấu trúc trong dữ liệu

...

30

30



4. Chuẩn hóa dữ liệu

Phương pháp chuẩn hóa dữ liệu

- Hướng tiếp cận cơ bản để chuẩn hóa như:

$$x_{new} = \frac{x_{old}}{x_{max}}$$

scale
(chia tỉ lệ)

[0;1]

$$x_{new} = \frac{x_{old} - x_{min}}{x_{max} - x_{min}}$$

min_max

$$x_{new} = \frac{x_{old} - \mu}{\sigma}$$

z-score

...

31

31



4. Chuẩn hóa dữ liệu

Phương pháp scale

	length	width	height
0	168.8	64.1	48.8
1	168.8	64.1	48.8
2	171.2	65.5	52.4
3	176.6	66.2	54.3
4	176.6	66.4	54.3
5	177.3	66.3	53.1
6	192.7	71.4	55.7
7	192.7	71.4	55.7
8	192.7	71.4	55.9

`df['length'] = df['length'] / df['length'].max()`

`df['width'] = df['width'] / df['width'].max()`

`df['height'] = df['height'] / df['height'].max()`

	length	width	height
0	0.811148	0.886584	0.816054
1	0.811148	0.886584	0.816054
2	0.822681	0.905947	0.876254
3	0.848630	0.915629	0.908027
4	0.848630	0.918396	0.908027
5	0.851994	0.917012	0.887960
6	0.925997	0.987552	0.931438
7	0.925997	0.987552	0.931438
8	0.925997	0.987552	0.934783

...

32

32



4. Chuẩn hóa dữ liệu

Phương pháp max_min

`df['length'] = (df['length'] - df['length'].min()) / (df['length'].max() - df['length'].min())`

`df['width'] = (df['width'] - df['width'].min()) / (df['width'].max() - df['width'].min())`

`df['height'] = (df['height'] - df['height'].min()) / (df['height'].max() - df['height'].min())`

	length	width	height
0	168.8	64.1	48.8
1	168.8	64.1	48.8
2	171.2	65.5	52.4
3	176.6	66.2	54.3
4	176.6	66.4	54.3
5	177.3	66.3	53.1
6	192.7	71.4	55.7

	length	width	height
0	0.413433	0.316667	0.083333
1	0.413433	0.316667	0.083333
2	0.449254	0.433333	0.383333
3	0.529851	0.491667	0.541667
4	0.529851	0.508333	0.541667
5	0.540299	0.500000	0.441667
6	0.770149	0.925000	0.658333

...

33

33



4. Chuẩn hóa dữ liệu

Phương pháp max_min (cách khác)

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

# Chuẩn hóa cột 'length' bằng Min-Max scaling
df['length2'] = scaler.fit_transform(df[['length']])
```

Ref.: <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler.html>

34

34



4. Chuẩn hóa dữ liệu

Phương pháp z-score

```
df['length'] = (df['length'] - df['length'].mean()) / df['length'].std()
df['width'] = (df['width'] - df['width'].mean()) / df['width'].std()
df['height'] = (df['height'] - df['height'].mean()) / df['height'].std()
```

	length	width	height		length	width	height
0	168.8	64.1	48.8	0	-0.425480	-0.842719	-2.015483
1	168.8	64.1	48.8	1	-0.425480	-0.842719	-2.015483
2	171.2	65.5	52.4	2	-0.230948	-0.190101	-0.542200
3	176.6	66.2	54.3	3	0.206750	0.136209	0.235366
4	176.6	66.4	54.3	4	0.206750	0.229440	0.235366
5	177.3	66.3	53.1	5	0.263488	0.182824	-0.255728
6	192.7	71.4	55.7	6	1.511737	2.560221	0.808309

35

35



4. Chuẩn hóa dữ liệu

Phương pháp z-score

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

# Chuẩn hóa cột 'length' bằng Z-score
df['length_Zscore'] = scaler.fit_transform(df[['length']])
```

36

36



4. Chuẩn hóa dữ liệu

Sử dụng statsmodels cho z-score

```
scipy.stats.zscore(a, axis=0, ddof=0, nan_policy='propagate')
```

Parameters:

- **a**: array_like. An array like object containing the sample data.
- **axis**: int or None, optional. Axis along which to operate. Default is 0. If None, compute over the whole array a.

Returns: zscore array_like

The z-scores, standardized by mean and standard deviation of input array a.

37

37



4. Chuẩn hóa dữ liệu

Áp dụng

```
1 scipy.stats.zscore(df[['length', 'width', 'height']])
```

```
array([[ -0.42652147, -0.84478235, -2.0204173 ],
       [ -0.42652147, -0.84478235, -2.0204173 ],
       [ -0.23151305, -0.19056612, -0.54352748],
       [  0.2072559 ,  0.13654199,  0.23594216],
       [  0.2072559 ,  0.23000146,  0.23594216],
       [  0.26413336,  0.18327172, -0.25635445],
       [  1.5154374 ,  2.56648799,  0.8102882 ],
       [  1.5154374 ,  2.56648799,  0.8102882 ],
       [  1.5154374 ,  2.56648799,  0.89233763],
       [  0.33726152,  0.93094742, -0.70762635],
```

38

38



5. Chuyển giá trị số thành các giá trị phân loại

Giới thiệu

- Giá trị phân loại (categorical variable)
- Nhóm các giá trị số thành các giá trị phân loại/(còn gọi là “bin”)
- Hay, gom nhóm các giá trị số thành một tập “bin”

Ví dụ:

Thuộc tính “price” có miền giá trị 5,000 đến 45,500

Được chuyển đổi thành các giá trị như “Thấp”, “Vừa”, “Cao”

price: 5000, 10000, 12000, 30000, 31000, 39000, 44000, 45000

Thấp Vừa Cao

39

39



5. Chuyển giá trị số thành các giá trị phân loại

Phương pháp:

- Bước #1: Tìm Max và Min
- Bước #2: Xác định số bin cần tạo (tức số lượng các giá trị phân loại)

Dùng:

- `numpy.linspace(start, stop, num=...)` => trả về list các cột mốc giá trị.
- `pandas.cut(x, bins, right=True, labels=None/Group, retbins=False, precision=3, include_lowest=False, duplicates='raise', ordered=True)`

	Bin 1	Bin 2	Bin 3	Bin 4	Bin 5	Bin 6	Bin 7	Bin 8	Bin 9	Bin 10
Range	0. to 3412.9	3412.9 to 6825.8	6825.8 to 10238.7	10238.7 to 13651.6	13651.6 to 17064.5	17064.5 to 20477.4	20477.4 to 23890.3	23890.3 to 27303.2	27303.2 to 30716.1	30716.1 to 34129.

...

40

40



5. Chuyển giá trị số thành các giá trị phân loại

Áp dụng

```
bins = np.linspace(min(df["price"]), max(df["price"]), 4)
```

```
group_names = ['Thấp', 'Vừa', 'Cao']
df['price-bin'] = pd.cut(df['price'], bins, labels=group_names,
                        include_lowest=True)
```

price	price-bin
7299	Thấp
22018	Vừa
8238	Thấp
34184	Cao
41315	Cao
16558	Thấp
6338	Thấp

...

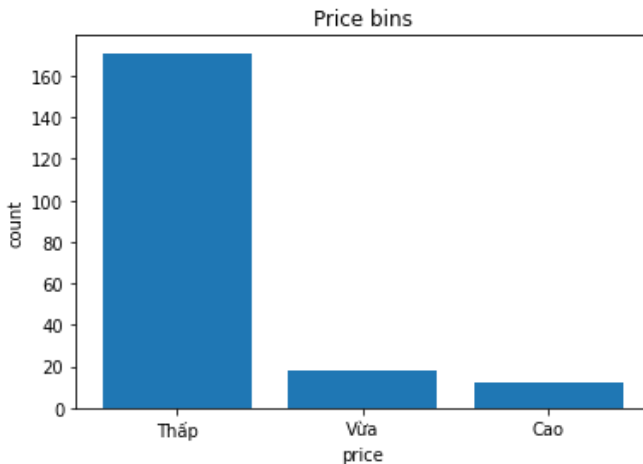
41

41



5. Chuyển giá trị số thành các giá trị phân loại

Trực quan các giá trị phân loại



```
1 %matplotlib inline
2 import matplotlib as plt
3 from matplotlib import pyplot
4 pyplot.bar(group_names,
5             df["price-bin"].value_counts())
6
7 plt.pyplot.xlabel("price")
8 plt.pyplot.ylabel("count")
9 plt.pyplot.title("Price bins")
```

42

42



6. Chuyển các giá trị phân loại thành giá trị số

Khó khăn?

- Các mô hình thống kê/ML không thể lấy biến là các kiểu **object/string**

fuel-type	drive-wheels	body-style	price
gas	fwd	hatchback	8949
gas	rwd	hatchback	18399
gas	fwd	wagon	8921
gas	fwd	wagon	14399
gas	fwd	wagon	8921
diesel	rwd	sedan	16900
gas	rwd	wagon	12440
gas	rwd	sedan	15580

Vì vậy, phải mã hóa (encode) sang dạng số để các mô hình phân tích có thể dùng được.

43

43



6. Chuyển các giá trị phân loại thành giá trị số

Giải pháp:

- Thêm các biến khác biểu diễn cho duy nhất mỗi giá trị phân loại.
- Chỉ định **0** hoặc **1** cho mỗi giá trị phân loại.
- Đây là phương pháp mã hóa “one-hot”, hoặc gọi là “one-hot encoding”

Name	Fuel		gas	diesel
A	gas		1	0
B	diesel		0	1
C	gas		1	0
D	gas		1	0

...

44

44



6. Chuyển các giá trị phân loại thành giá trị số

Kỹ thuật

Signature:

```
pd.get_dummies(  
    data,  
    prefix=None,  
    prefix_sep='_',  
    dummy_na=False,  
    columns=None,  
    sparse=False,  
    drop_first=False,  
    dtype=None,  
) -> 'DataFrame'
```

Docstring:

Convert categorical variable into dummy/indicator variables.

...

45

45



6. Chuyển các giá trị phân loại thành giá trị số

Áp dụng

- Dùng phương thức `pandas.get_dummies()`
- Chuyển giá trị phân loại sang giá trị số (**0**, hoặc **1**)

fuel-type
gas
gas
gas
gas
diesel
gas

```
pd.get_dummies(df['fuel-type'])
```

diesel	gas
0	1
0	1
0	1
0	1
1	0
0	1

46

46



6. Chuyển các giá trị phân loại thành giá trị số

Phương pháp khác

- Integer Encoding: ánh xạ thành các con số
- Frequency Encoding: gán giá trị số dựa trên tần suất xuất hiện
- Ordinal Encoding: biến phân loại có thứ tự => thì có thể ánh xạ thành các giá trị số theo thứ tự
- Mapping

Ref.: <https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Categorical.html#pandas.Categorical>

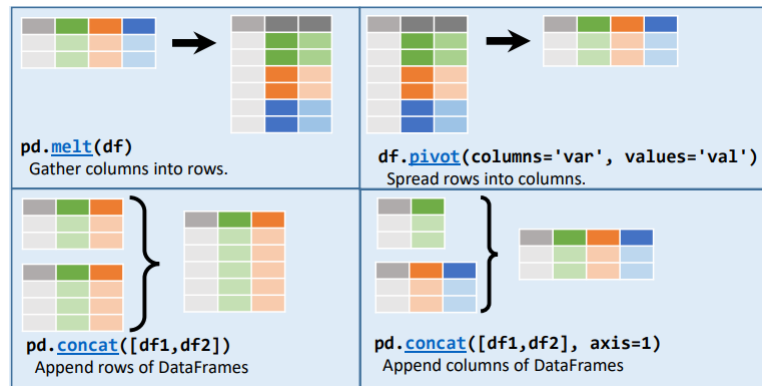
47

47



Tóm tắt

Ref.: https://pandas.pydata.org/Pandas_Cheat_Sheet.pdf



48

48



50



Hỏi - Đáp



...

51